========== Step 1: Importing Required libraries ==========

In [1]:
```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, classifi
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.utils import plot_model
import joblib, os
import seaborn as sns
from sklearn.preprocessing import label_binarize
from sklearn.metrics import roc_curve, auc
from itertools import cycle
import matplotlib.pyplot as plt
```

========== Step 2: Load dataset ==========

In [2]:
```
# Load dataset
df = pd.read_csv('Dataset/heart_cleveland_upload.csv')
df
```

Out[2]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	condition
0	69	1	0	160	234	1	2	131	0	0.1	1	1	0	0
1	69	0	0	140	239	0	0	151	0	1.8	0	2	0	0
2	66	0	0	150	226	0	0	114	0	2.6	2	0	0	0
3	65	1	0	138	282	1	2	174	0	1.4	1	1	0	1
4	64	1	0	110	211	0	2	144	1	1.8	1	0	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
292	40	1	3	152	223	0	0	181	0	0.0	0	0	2	1
293	39	1	3	118	219	0	0	140	0	1.2	1	0	2	1
294	35	1	3	120	198	0	0	130	1	1.6	1	0	2	1
295	35	0	3	138	183	0	0	182	0	1.4	0	0	0	0
296	35	1	3	126	282	0	2	156	1	0.0	0	0	2	1

297 rows × 14 columns

========== Step 3: Pre-processing ==========

In [3]:
```
df.shape
```

Out[3]:
```
(297, 14)
```

In [4]:
```
df.columns
```

Out[4]:
```
Index(['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach',
       'exang', 'oldpeak', 'slope', 'ca', 'thal', 'condition'],
      dtype='object')
```

In [5]:
```
df.isnull().sum()
```

Out[5]:
```
age          0
sex          0
cp           0
trestbps     0
chol         0
fbs          0
restecg      0
thalach      0
exang        0
oldpeak      0
slope        0
ca           0
thal         0
condition    0
dtype: int64
```

In [6]:
```
df.describe()
```

Out[6]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	2
count	297.000000	297.000000	297.000000	297.000000	297.000000	297.000000	297.000000	297.000000	297.000000	297.000000	
mean	54.542088	0.676768	2.158249	131.693603	247.350168	0.144781	0.996633	149.599327	0.326599	1.055556	
std	9.049736	0.468500	0.964859	17.762806	51.997583	0.352474	0.994914	22.941562	0.469761	1.166123	
min	29.000000	0.000000	0.000000	94.000000	126.000000	0.000000	0.000000	71.000000	0.000000	0.000000	
25%	48.000000	0.000000	2.000000	120.000000	211.000000	0.000000	0.000000	133.000000	0.000000	0.000000	
50%	56.000000	1.000000	2.000000	130.000000	243.000000	0.000000	1.000000	153.000000	0.000000	0.800000	
75%	61.000000	1.000000	3.000000	140.000000	276.000000	0.000000	2.000000	166.000000	1.000000	1.600000	
max	77.000000	1.000000	3.000000	200.000000	564.000000	1.000000	2.000000	202.000000	1.000000	6.200000	

In [7]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 297 entries, 0 to 296
Data columns (total 14 columns):
age          297 non-null int64
sex          297 non-null int64
cp           297 non-null int64
trestbps     297 non-null int64
chol         297 non-null int64
fbs          297 non-null int64
restecg      297 non-null int64
thalach      297 non-null int64
exang        297 non-null int64
oldpeak      297 non-null float64
slope        297 non-null int64
ca           297 non-null int64
thal         297 non-null int64
condition    297 non-null int64
dtypes: float64(1), int64(13)
memory usage: 32.6 KB
```

In [8]:
```
df.duplicated().sum()
```

Out[8]:
```
0
```

========== Step 4: Splitting ==========

In [9]:
```
# Split into features and target
X = df.drop('condition', axis=1)
y = df['condition']

# Normalize the features
#scaler = StandardScaler()
#X_scaled = scaler.fit_transform(X)

# Split into train-test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

In [10]:
```
X_train
```

Out[10]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal
273	46	1	3	140	311	0	0	120	1	1.8	1	2	2
259	51	0	3	130	305	0	0	142	1	1.2	1	0	2
30	59	1	1	140	221	0	0	164	1	0.0	0	0	0
22	34	1	0	118	182	0	2	174	0	0.0	0	0	0
277	45	1	3	142	309	0	2	147	1	0.0	1	3	2
...	...	...	...	...	...	...	...	...	...	...	...	...	...
188	62	0	3	140	394	0	2	157	0	1.2	1	0	0
71	29	1	1	130	204	0	2	202	0	0.0	0	0	0
106	56	1	2	130	256	1	2	142	1	0.6	1	1	1
270	47	1	3	110	275	0	2	118	1	1.0	1	1	0
102	58	1	2	140	211	1	2	165	0	0.0	0	0	0

237 rows × 13 columns

In [11]:
```
y_train
```

Out[11]:
```
273    1
259    1
30     0
22     0
277    1
      ..
188    0
71     0
106    1
270    1
102    0
Name: condition, Length: 237, dtype: int64
```

In [12]:
```
# Global metric tracking
metrics_df = pd.DataFrame(columns=['Algorithm', 'Accuracy', 'Precision', 'Recall', 'F1-Score'])
MODEL_DIR = 'models'
os.makedirs(MODEL_DIR, exist_ok=True)
categories = ['No disease', 'Disease']
def Calculate_Metrics(algorithm, y_pred, y_test, y_score=None):
    #le_target = joblib.load(os.path.join(MODEL_DIR, 'target_label_encoder1.pkl'))
    #categories = le_target.classes_

    # Metrics
    acc = accuracy_score(y_test, y_pred) * 100
    prec = precision_score(y_test, y_pred, average='macro') * 100
    rec = recall_score(y_test, y_pred, average='macro') * 100
    f1 = f1_score(y_test, y_pred, average='macro') * 100

    metrics_df.loc[len(metrics_df)] = [algorithm, acc, prec, rec, f1]

    print(f"{algorithm} Accuracy: {acc:.2f}%")
    print(f"{algorithm} Precision: {prec:.2f}%")
    print(f"{algorithm} Recall: {rec:.2f}%")
    print(f"{algorithm} F1-Score: {f1:.2f}%")
   # print(f"\nClassification Report:\n{classification_report(y_test, y_pred, target_names=categories)}")
    print(f"\nClassification Report:\n{classification_report(y_test, y_pred, labels=range(len(categories)), tar
    # Confusion matrix
    cm = confusion_matrix(y_test, y_pred)
    plt.figure(figsize=(8, 6))
    sns.heatmap(cm, annot=True, xticklabels=categories, yticklabels=categories, cmap="Blues", fmt="g")
    plt.title(f'{algorithm} - Confusion Matrix')
    plt.ylabel('True Label')
    plt.xlabel('Predicted Label')
    plt.savefig(f"results/{algorithm.replace(' ', '_')}_confusion_matrix.png")
    plt.show()

    # ROC Curve Plot (if y_score is provided)
    if y_score is not None:
        y_test_bin = label_binarize(y_test, classes=range(len(categories)))

        plt.figure(figsize=(10, 8))

        # Handle Binary Classification Case
        if len(categories) == 2:
            # Use probability of positive class (1)
            fpr, tpr, _ = roc_curve(y_test, y_score[:, 1])
            roc_auc = auc(fpr, tpr)
            plt.plot(fpr, tpr, label=f'{categories[1]} vs {categories[0]} (AUC = {roc_auc:.2f})')
```

```
        # Handle Multi-Class Case
        else:
            fpr = dict()
            tpr = dict()
            roc_auc = dict()
            for i in range(len(categories)):
                fpr[i], tpr[i], _ = roc_curve(y_test_bin[:, i], y_score[:, i])
                roc_auc[i] = auc(fpr[i], tpr[i])
                plt.plot(fpr[i], tpr[i], label=f'Class {categories[i]} (AUC = {roc_auc[i]:.2f})')

        plt.plot([0, 1], [0, 1], 'k--', label='Random')
        plt.title(f"{algorithm} ROC Curve")
        plt.xlabel('False Positive Rate')
        plt.ylabel('True Positive Rate')
        plt.legend(loc='lower right')
        plt.grid(True)
        plt.tight_layout()
        plt.savefig(f"results/{algorithm.replace(' ', '_')}_roc_curve.png")
        plt.show()
```

========== Step 5: Build ANN Model ==========

In [13]:
```
model = Sequential([
    Dense(64, activation='relu', input_shape=(X_train.shape[1],)),
    Dense(32, activation='relu'),
    Dense(1, activation='sigmoid')
])
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# Train
model.fit(X_train, y_train, epochs=100, batch_size=16, validation_split=0.1, verbose=0)

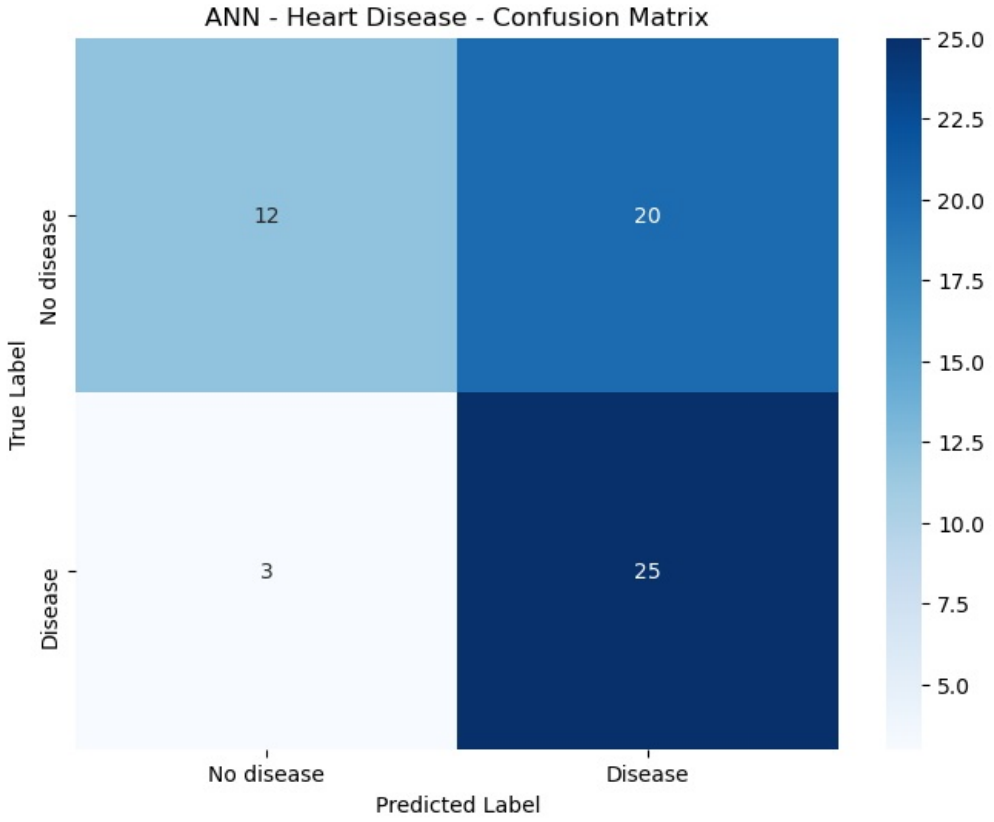
# Predict
y_prob = model.predict(X_test)
y_pred = (y_prob > 0.5).astype("int32").flatten()
Calculate_Metrics("ANN - Heart Disease", y_pred, y_test, y_score=np.hstack([1 - y_prob, y_prob]))
```

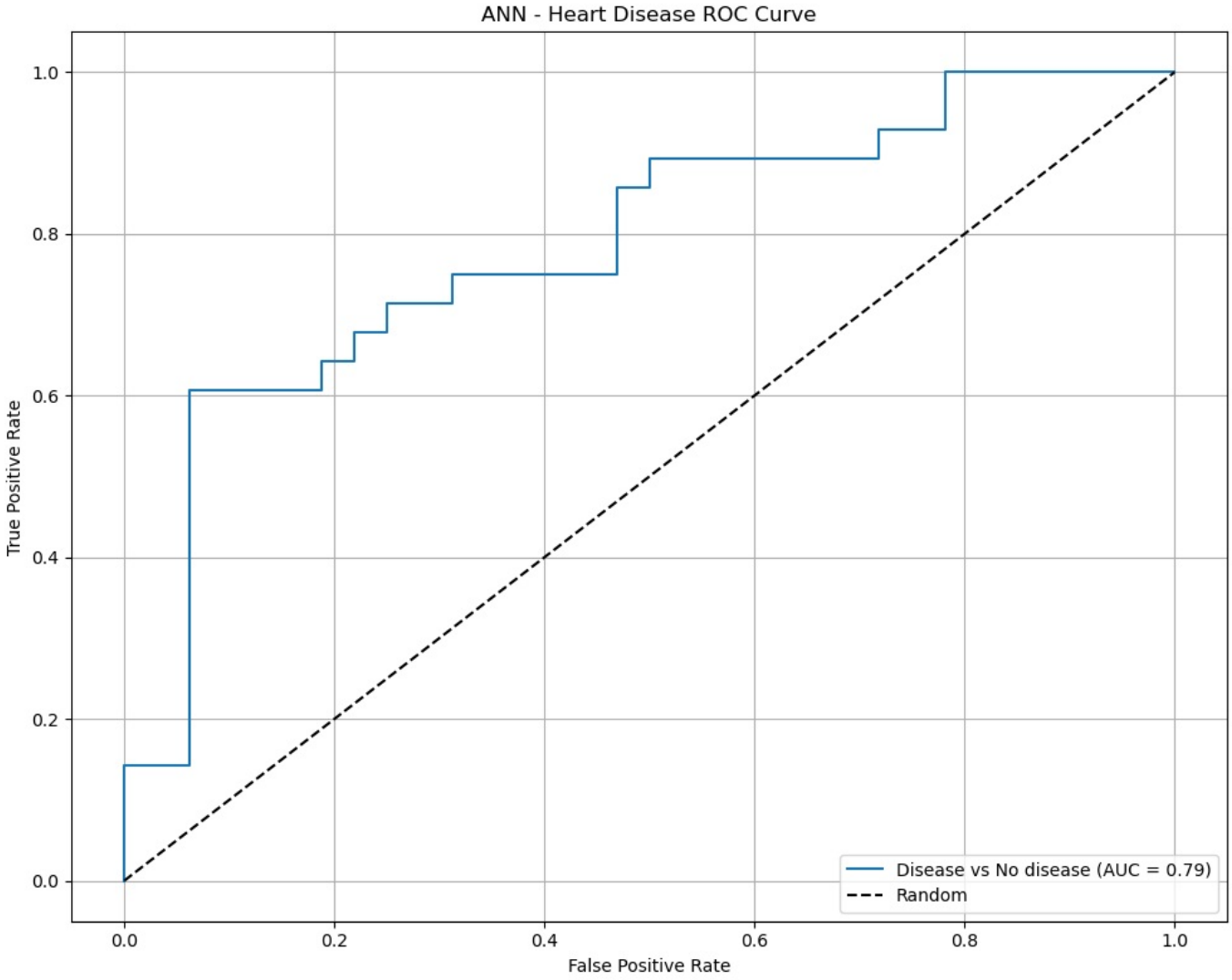
```
WARNING:tensorflow:Falling back from v2 loop because of error: Failed to find data adapter that can handle input
: <class 'pandas.core.frame.DataFrame'>, <class 'NoneType'>
WARNING:tensorflow:Falling back from v2 loop because of error: Failed to find data adapter that can handle input
: <class 'pandas.core.frame.DataFrame'>, <class 'NoneType'>
ANN - Heart Disease Accuracy: 61.67%
ANN - Heart Disease Precision: 67.78%
ANN - Heart Disease Recall: 63.39%
ANN - Heart Disease F1-Score: 59.78%

Classification Report:
              precision    recall  f1-score   support

  No disease       0.80      0.38      0.51        32
     Disease       0.56      0.89      0.68        28

    accuracy                           0.62        60
   macro avg       0.68      0.63      0.60        60
weighted avg       0.69      0.62      0.59        60
```


ANN - Heart Disease - Confusion Matrix
12
20
3
25
No disease
Disease
True Label
No disease
Disease
Predicted Label
25.0
22.5
20.0
17.5
15.0
12.5
10.0
7.5
5.0



========== Step 6: Prediction ==========

In [14]:
```
test_data = df.sample(20)
test_data
```

Out[14]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	condition
59	44	1	1	120	263	0	0	173	0	0.0	0	0	2	0
32	58	0	1	136	319	1	2	152	0	0.0	0	2	0	1
57	45	0	1	112	160	0	0	138	0	0.0	1	0	0	0
8	60	0	0	150	240	0	0	171	0	0.9	0	0	0	0
100	58	1	2	132	224	0	2	173	0	3.2	0	2	2	1
98	58	1	2	112	230	0	2	165	0	2.5	1	1	2	1
250	53	1	3	140	203	1	2	155	1	3.1	2	0	2	1
193	61	0	3	145	307	0	2	146	1	1.0	1	0	2	1
119	51	1	2	125	245	1	2	166	0	2.4	1	0	0	0
9	59	1	0	178	270	0	2	145	0	4.2	2	0	2	0
293	39	1	3	118	219	0	0	140	0	1.2	1	0	2	1
144	42	1	2	120	240	1	0	194	0	0.8	2	0	2	0
14	56	1	0	120	193	0	2	162	0	1.9	1	0	2	0
276	45	0	3	138	236	0	2	152	1	0.2	1	0	0	0
230	57	1	3	130	131	0	0	115	1	1.2	1	1	2	1
159	68	1	3	144	193	1	0	141	0	3.4	1	2	2	1
107	54	1	2	125	273	0	2	152	0	0.5	2	1	0	0
126	50	1	2	140	233	0	0	163	0	0.6	1	1	2	1
284	43	1	3	115	303	0	0	181	0	1.2	1	0	0	0
182	63	0	3	124	197	0	0	136	1	0.0	1	0	0	1

In [15]:
```
test_data.drop('condition', inplace = True, axis = 1)
```

In [16]:
```
test_data
```

Out[16]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal
59	44	1	1	120	263	0	0	173	0	0.0	0	0	2
32	58	0	1	136	319	1	2	152	0	0.0	0	2	0
57	45	0	1	112	160	0	0	138	0	0.0	1	0	0
8	60	0	0	150	240	0	0	171	0	0.9	0	0	0
100	58	1	2	132	224	0	2	173	0	3.2	0	2	2
98	58	1	2	112	230	0	2	165	0	2.5	1	1	2
250	53	1	3	140	203	1	2	155	1	3.1	2	0	2
193	61	0	3	145	307	0	2	146	1	1.0	1	0	2
119	51	1	2	125	245	1	2	166	0	2.4	1	0	0
9	59	1	0	178	270	0	2	145	0	4.2	2	0	2
293	39	1	3	118	219	0	0	140	0	1.2	1	0	2
144	42	1	2	120	240	1	0	194	0	0.8	2	0	2
14	56	1	0	120	193	0	2	162	0	1.9	1	0	2
276	45	0	3	138	236	0	2	152	1	0.2	1	0	0
230	57	1	3	130	131	0	0	115	1	1.2	1	1	2
159	68	1	3	144	193	1	0	141	0	3.4	1	2	2
107	54	1	2	125	273	0	2	152	0	0.5	2	1	0
126	50	1	2	140	233	0	0	163	0	0.6	1	1	2
284	43	1	3	115	303	0	0	181	0	1.2	1	0	0
182	63	0	3	124	197	0	0	136	1	0.0	1	0	0

In [17]:
```
#test_data_scaled = scaler.transform(test_data)
test_preds = model.predict(test_data)
final_preds = (test_preds > 0.5).astype(int).flatten()
print("Predictions:\n", final_preds)
```

```
WARNING:tensorflow:Falling back from v2 loop because of error: Failed to find data adapter that can handle input
: <class 'pandas.core.frame.DataFrame'>, <class 'NoneType'>
Predictions:
 [1 1 0 0 1 1 1 1 1 1 1 0 1 1 1 1 1 1 0 0]
```

In [18]:
```
test_data["Prediction"] = [categories[i] for i in final_preds]
test_data
```

Out[18]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	Prediction
59	44	1	1	120	263	0	0	173	0	0.0	0	0	2	Disease
32	58	0	1	136	319	1	2	152	0	0.0	0	2	0	Disease
57	45	0	1	112	160	0	0	138	0	0.0	1	0	0	No disease
8	60	0	0	150	240	0	0	171	0	0.9	0	0	0	No disease
100	58	1	2	132	224	0	2	173	0	3.2	0	2	2	Disease
98	58	1	2	112	230	0	2	165	0	2.5	1	1	2	Disease
250	53	1	3	140	203	1	2	155	1	3.1	2	0	2	Disease
193	61	0	3	145	307	0	2	146	1	1.0	1	0	2	Disease
119	51	1	2	125	245	1	2	166	0	2.4	1	0	0	Disease
9	59	1	0	178	270	0	2	145	0	4.2	2	0	2	Disease
293	39	1	3	118	219	0	0	140	0	1.2	1	0	2	Disease
144	42	1	2	120	240	1	0	194	0	0.8	2	0	2	No disease
14	56	1	0	120	193	0	2	162	0	1.9	1	0	2	Disease
276	45	0	3	138	236	0	2	152	1	0.2	1	0	0	Disease
230	57	1	3	130	131	0	0	115	1	1.2	1	1	2	Disease
159	68	1	3	144	193	1	0	141	0	3.4	1	2	2	Disease
107	54	1	2	125	273	0	2	152	0	0.5	2	1	0	Disease
126	50	1	2	140	233	0	0	163	0	0.6	1	1	2	Disease
284	43	1	3	115	303	0	0	181	0	1.2	1	0	0	No disease
182	63	0	3	124	197	0	0	136	1	0.0	1	0	0	No disease

In [ ]:

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js